

MANEJO DE TRANSFORMACIONES: POSICIÓN, ROTACIÓN Y ESCALA

1. Haz que una esfera en escena se posicione en la posición (5, 3, 5) al iniciar el juego.
2. Haz que un cubo, se posicione en un punto random al comenzar el juego, cada coordenada entre -10 y 10, ambos inclusive. Si se mantiene pulsada alguna tecla 'WASD', el cubo se ha de posicionar a 5 puntos desde su posición inicial en el eje correspondiente de la tecla (adelante, atrás, izquierda, derecha, respectivamente). Si no hay ninguna tecla 'WASD' mantenida, entonces se vuelve a posicionar en la posición inicial.
3. Haz que un cubo, inicialmente con una rotación random en Z (de 0° a 360°) (X e Y en 0), rote +15/-15 grados en su eje Z, por cada vez que se pulsan las teclas 'D'/'A', respectivamente.
4. Haz que una esfera con escala (1, 1, 1), se escale a 1 unidades más en todos sus ejes cada vez que se pulsa el espacio. Si la esfera llega a escala (10, 10, 10), entonces, reseteará la escala a la original.
5. Crea un gameObject vacío con un script "Manejador". Desde sólo dicho script, referencia a una esfera y a un cubo puestos en escena. Haz que el cubo se posicione en un punto random (-10, 10 en todos los ejes) y la esfera a 3 unidades más en el eje X respecto al cubo. También, haz que el radio del SphereCollider de la esfera sea del mismo tamaño que la posición en la coordenada Y del cubo.
6. Crea un gameObject vacío con un script "Manejador". Desde sólo dicho script, referencia a dos esferas puestas en escena que parten desde el punto (0, 0, 0). Haz que, si se mantiene el espacio presionado, la esfera A avance 0.001 unidades en el eje X y que la esfera B retroceda 0.001 unidades en el eje X, de tal manera que se vayan separando. Si en algún momento, alguna esfera excede en su coordenada X 10 ó -10, esta cambiará de dirección. Es decir, si estaba retrocediendo pasará a avanzar y viceversa, volviéndose a encontrar una con la otra. Deshabilitar el componente MeshRenderer de las esferas si estas se encuentran en el rango de 5 a 7 ó de -5 a -7 en X. Volviendo a habilitar estos componentes en el resto del trayecto.

MOVIMIENTOS CINEMÁTICOS

1. Haz que un cubo se traslade por sí solo en el eje Z global positivo a una velocidad de 2 u/s. Además de eso, haz que el cubo rote en su eje X local positivo/negativo a una velocidad de 90°/s a partir de la lectura de la tecla 'eje' horizontal.
2. Haz que un cubo se traslade **por sí solo** en su eje Z local positivo a una velocidad de 3 u/s a medida que rota **por sí solo** en su eje X a 45°/s. ¿Por qué realiza un comportamiento extraño? ¿Cómo lo solucionarías?
3. Haz que un cubo se traslade en todas las direcciones posibles que conforman sus ejes locales X, Z a una velocidad de 5 u/s a partir de las teclas 'eje' (Axis)
4. Haz que un cubo se traslade inicialmente por sí solo de forma continuada en su eje X local positivo a una velocidad de 3 u/s. Haz que la lectura de las teclas eje modifiquen su dirección, **sólo su dirección**. Es decir, **por ejemplo**, si en algún momento es presionada la tecla 'A', la dirección de movimiento pasará a ser su eje X local negativo, sin necesidad de seguir manteniendo la tecla.

5. Haz que una cápsula se reescale 2 u/s en cada uno de sus ejes. Si la cápsula adquiere una escala en todos sus ejes de 10u, entonces se reescalará a la misma velocidad pero en el sentido contrario. Es decir, se comprime, se ensancha, se comprime, se ensancha, etc.
6. Haz que un cubo inicialmente posicionado en (0, 0, 0) se traslade en el eje Y global a partir de la lectura de las teclas 'eje' verticales a una velocidad de 5 u/s. El movimiento se debe ver limitado: El cubo no podrá pasar de la coordenada Y= 10 ó Y = -10.
7. Haz que un cubo inicialmente posicionado en un punto **Random**, se traslade en un eje **unitario Random** a una velocidad **Random** de 3 a 6 u/s (ambos inclusive). Si el cubo recorre **una distancia** de 15 u, se debe detener y no seguirá trasladándose.

MÉTODO UPDATE E INPUTS

1. Programa que aumente una variable llamada **contador** de uno en uno por cada vez que se hace click izquierdo y disminuya de uno en uno cada vez que se hace click derecho. El contador no podrá quedarse bajo ningún concepto en un número negativo, como mínimo en 0. Sacar mensajes de consola para comprobar el proceso.
2. Programa el comportamiento básico de un revólver en un juego de Survival. El revólver dispara una bala cada vez que hacemos click izquierdo. Su capacidad **máxima** es de 7 balas y podemos recargar en cualquier momento haciendo click derecho. No podremos disparar si no tenemos balas. No podremos recargar si ya tenemos 7 balas. Muestra mensajes por consola para comprobar el correcto funcionamiento.
3. Programa el comportamiento básico de una metralleta en un juego de Survival. Es el mismo comportamiento que la pistola, pero con 40 balas, las cuales se disparan **manteniendo** el click izquierdo. La recarga funciona igual.
4. Programa el comportamiento básico de un **salto cargado** de un juego tipo plataformas. A medida que se **mantiene presionado** el espacio, se va cargando una variable **fuerzaSalto**. No puede llegar a más de 300. Si el espacio es **levantado**, saltaremos con dicha fuerza acumulada y fuerzaSalto ha de ser **resetada** para saltos posteriores.
5. Programa los Inputs básicos de movimiento para un personaje que tiene velocidad 5. Puede desplazarse tanto con WASD como con las flechas y podrá saltar al pulsar el espacio. Además:
 - a. Se mantendrá agachado manteniendo el CTRL izquierdo y reduciendo su velocidad a la mitad. Si se levanta, la velocidad se reseteará a la original.
 - b. **Sólo si** hay movimiento, manteniendo el SHIFT podrá correr, aumentando su velocidad en un 150%. Si se levanta, la velocidad se reseteará a la original.
 - c. **El agacharse precede al correr**. Es decir, si estamos corriendo y mantenemos CTRL izquierdo, hemos de agacharnos. En cambio, si estamos agachados y mantenemos el SHIFT, no podremos correr.

REFERENCIA A GAMEOBJECTS Y COMPONENTES

1. Desde un script de una esfera llamada "Bola" realiza los siguientes comportamientos:
 - a. Si pulsamos al espacio, aparecerá por consola el nombre de este gameObject.
 - b. Si pulsamos a la 'Z', se deshabilitará su MeshRenderer. Si ya está deshabilitado, entonces lo destruirá.
 - c. Si pulsamos a la 'X', se deshabilitará su SphereCollider. Si ya está deshabilitado, entonces lo destruirá.
 - d. Si pulsamos a la 'C', el gameObject al completo será destruido.
2. Desde el script de un gameObject A, referencia a un gameObject B y haz que este cambie de estado (se habilite/deshabilite) cada vez que pulsemos el espacio.
3. Desde el script de un gameObject A, referencia a un gameObject B y haz que se deshabilite su collider cuando pulsemos el espacio.
4. Desde el script de un gameObject A, referencia directamente el MeshRenderer de un gameObject B por el Inspector y haz que este cambie de estado (se habilite/deshabilite) por cada 3 pulsaciones de la tecla Escape.
5. Crea un cubo y una esfera simulando un busto de un personaje. Llama al cubo "Busto". Haz la esfera hija del cubo. Crea 10 réplicas de este gameObject Personaje y espárcelos por la escena.
 - a. Desde el script de otro gameObject cualquiera, referencia estos 10 gameObjects "Busto", creando una colección. Realiza un código de tal manera que, cada vez que pulsemos el espacio, se vayan deshabilitando el componente MeshRenderer de las cabezas de los bustos.
 - b. Es decir, si pulsamos una vez, se deshabilitará el MeshRenderer de la primera cabeza, si pulsamos una segunda, el de la segunda.
 - c. Si se llegan a deshabilitar las 10 cabezas, se "reseteará" el comportamiento, recorriendo de nuevo todas las cabezas pero esta vez, habilitándolas.
 - d. El comportamiento será cíclico y sin fin.
6. Dado un gameObject vacío con un script y dos esferas (A y B) inicialmente deshabilitadas. Desde el script del gameObject vacío, referenciar el MeshRenderer de la esfera A y el Collider de la esfera B.
 - a. Desde este mismo script, se ha de poder destruir la esfera 'A' al pulsar la tecla 'A' y destruir la esfera B al pulsar la tecla 'B'.
 - b. ¿Por qué da error si pulsamos varias veces? Buscar una forma para prevenir el error.

MOVIMIENTOS DINÁMICOS. RIGIDBODIES, ADDFORCE y MATERIALES FÍSICOS.

1. Dado un cubo sobre ninguna superficie con un Rigidbody, este ha de cambiar de estado (de cinemático a dinámico y viceversa) cada vez que se pulsa el espacio. ¿Por qué cae al estar en dinámico? Coméntalo en alguna parte del código.
2. Nada más comenzar el juego, se ha de añadir una fuerza tipo impulso de 5u en el eje X global positivo a un cubo inicialmente quieto apoyado en una superficie plana con un Rigidbody dinámico. Ahora prueba a añadirle la fuerza pero cuando

pulsemos el espacio. En ambos casos, ¿hace falta utilizar el FixedUpdate? ¿Por qué?

3. Haz que una esfera se mueva dinámicamente a partir de su Rigidbody en función de la lectura de las teclas eje. La esfera se iniciará apoyada sobre una superficie plana y cuatro paredes. La fuerza de movimiento será una fuerza continua de 5u en la dirección apropiada mientras alguna de estas teclas sea mantenida. También, haz que la esfera pueda saltar cada vez que se pulsa el espacio a partir de una fuerza tipo impulso en el eje global Y positivo de 10u de fuerza. ¿En qué casos hace falta utilizar el FixedUpdate? ¿Por qué? ¿Por qué aún habiendo dejado de pulsar las teclas de movimiento la esfera sigue moviéndose?
4. Simula un cañón con un cubo con escala (0.5, 0.5, 2) y reposiciona su pivote para que este se encuentre en el “pie” del cañón. Después, crea una esfera con escala (0.3, 0.3, 0.3) con rigidbody cinemático y sitúala en la boca del cañón y emparentala a este. Mueve la cámara para que esta tome una vista lateral del cañón. Desde un script que lleve el cañón, consigue que este rote cinemáticamente desde su pie a partir de las teclas ‘eje’ horizontales (‘A’, ‘D’ y flechas). También, desde este script, referencia a la bola.
5. Si el espacio es mantenido, se va **acumulando** una fuerza de 0.5 u **por segundo**. Una vez se suelte el espacio, la bola se ha de **desemparentar** del cañón (método **SetParent()** de la clase **Transform**), ha de pasar a **dinámico** y ha de salir **impulsada** con dicha fuerza acumulada.
6. Estira un cubo a modo de plataforma, y posiciona encima de esta a dos esferas de 1u de radio con rigidbodies a lo largo del eje Z global (esfera A y esfera B) de tal manera que si A se mueve en el eje Z se acaben encontrando y realiza las siguientes configuraciones:
 - a. Esfera A en cinemático, esfera B en dinámico, con **masa infinita**. Haz que la esfera A se traslade en dirección eje Z global. ¿Por qué ocurre dicho comportamiento? Apúntalo en comentarios de código.
 - b. Esfera A en cinemática, esfera B en cinemático. Haz que la esfera A se traslade en dirección eje Z global. ¿Por qué ocurre dicho comportamiento? Apúntalo en comentarios de código.
 - c. Esfera A en dinámico, esfera B en dinámico, **masa 0.2**. Aplica una fuerza tipo impulso a la esfera A en dirección eje Z global. ¿Por qué ocurre dicho comportamiento? Apúntalo en comentarios de código.
 - d. Esfera A en dinámico, esfera B en cinemático. Aplica una fuerza tipo impulso a la esfera A en dirección eje Z global. ¿Por qué ocurre dicho comportamiento? Apúntalo en comentarios de código.

INTERACCIONES: TIPO TRIGGER Y COLLISION

1. Haz que una esfera que se mueve de forma cinética con las teclas eje, destruya un cubo inicialmente quieto en cuanto le comience a atravesar.
2. Añade más gameObjects en escena (cápsulas, otras esferas, cilindros, etc) y haz que la esfera sólo destruya a los gameObjects etiquetados con tag “Cubo”.
3. Manteniendo el movimiento cinético de la esfera, intenta destruir los gameObjects etiquetados con tag “Cubo” al **colisionar** con ellos. ¿Por qué no funciona?

4. Cambia el Rigidbody de la esfera a dinámico, haz que esta se mueva por fuerzas a partir de las teclas eje con una fuerza elevada (por ejemplo, 20u). Intenta destruir los gameObjects etiquetados con tag "Cubo" al colisionar con ellos. ¿Qué diferencia ves respecto a la interacción tipo Trigger del ejercicio 14.1?
5. Haz que también se destruya la propia bola si esta colisiona con algún gameObject etiquetado con tag "Cubo".
6. Haz que una esfera movida con fuerza de 10u en función de las teclas eje se mueva a la mitad de la fuerza original (5u) mientras permanezca dentro de cierta zona (por ejemplo "lava"). Al salir de la zona de lava, ha de volver a moverse con la fuerza original de 5u.
7. Apila varios "muros" invisibles a partir de un cubo alargado sin MeshRenderer a lo largo del eje X positivo. Por ejemplo, en (2, 0, 0), (4, 0, 0), (6, 0, 0), etc. Haz que una esfera que parte desde (0, 0, 0) se mueva por sí sola cinéticamente en el eje global X positivo. Cuando entre a alguno de estos muros, se ha de deshabilitar su componente MeshRenderer y se volverá a habilitar al salir de estos.
8. Haz que un cubo con un rigidbody dinámico inicialmente apoyado sobre una superficie se pueda mover en función de las teclas eje con una fuerza continua de 25u. Dispón debajo de la superficie una "zona de muerte" (cubo invisible). Si el cubo llega a atravesar dicha zona ha de ser destruido.

CONCEPTOS 2D Y COMPONENTE SPRITERENDERER

1. Dado un sprite de un personaje cualquiera, consigue que este se desplace de forma cinemática sólo en el eje 'X' a partir de las teclas eje. El sprite se ha de voltear en la dirección adecuada. Es decir, si existe desplazamiento hacia la derecha, el sprite mirará hacia la derecha y lo mismo con la izquierda.
2. Se pretende simular el cambio de sprites de un cerdo en un juego tipo Angry Birds. Para ello, haz que un sprite que represente el pájaro salga impulsado con una fuerza en dirección al cerdo. Cuando el pájaro colisione con el cerdo, este cambiará a un sprite diferente.
3. Completa el ejercicio 15.1. de tal manera que nuestro personaje cambie a color rojo cuando entre en una zona de "peligro" (otro gameObject representado por otro sprite). Cuando el personaje salga de la zona de "peligro" volverá a su color blanco original.
4. Dado un sprite de un personaje cualquiera apoyado sobre una superficie, consigue que este se mueva de forma dinámica en la horizontal en función de la lectura de las teclas 'eje' horizontales. También, consigue que se voltee en la dirección adecuada.
5. Haz que el personaje del ejercicio anterior pueda saltar por cada vez que pulsemos el espacio sólo cuando esté sobre el suelo.
6. Consigue que el salto implementado en el apartado anterior quede lo más fluido posible alterando el campo "Gravity Scale" del Rigidbody2D y la fuerza del salto.

CONCEPTO DE PREFAB E INSTANCIACIONES.

1. Dado un gameObject vacío con un script llamado Spawner, se ha de instanciar una copia de un prefab cualquiera en la posición (0, 0, 0) con rotación (0, 0, 0)

2. Dado un gameObject vacío con un script llamado Spawner, se ha de instanciar una copia de un prefab cuadrado. Dicho cuadrado, a su vez, nada más aparecer, instanciará una copia de un prefab círculo.
3. Se pretende desarrollar el comportamiento de una nave de un juego tipo Space Shooter. Para ello, haz que una nave se traslade de forma cinética por sí sola en dirección X negativo global a 5u/s. Cada vez que pulsemos el espacio, esta nave instanciará una copia de un prefab “disparo”. Dichos disparos se han de trasladar por sí solos a 5 u/s en el eje global X negativo. A su vez, cuando un “disparo” sale a escena instanciará dos “subdisparos”, los cuales se trasladarán a 5u/s en las diagonales del eje global X negativo.
4. Dado un gameObject vacío con un script llamado Spawner, se han de instanciar 50 copias de un prefab cuadrado nada más iniciar el juego. Estos cuadrados se instanciarán en una posición X random (entre -50 y 50) e Y random (entre -50 y 50) siendo Z =0. También se instanciarán con rotación random (entre 0 a 360°).
5. Completa el ejercicio anterior de tal manera que, cada cuadrado que sale a escena, instancie a 2.5 unidades por encima de ellos una copia de un prefab de círculo.

MINIJUEGOS

1. Se pretende desarrollar un juego tipo Space Shooter que funciona por oleadas. Habrá un total de 10 niveles, de 3 oleadas cada nivel, de 10 enemigos por oleada.
 - a. Habrá una espera de 3 segundos antes de empezar cada nivel, una espera de 2 segundos antes de empezar cada oleada y una espera de 1 segundo entre cada instancia de enemigo.
 - b. Puedes llevar la cuenta del nivel, oleada y número de enemigo por el que va tu juego a través de mensajes por consola con Debug.Log
 - c. Las naves que se van instanciando se trasladarán en el eje X negativo global a una velocidad de 5 u/s. A su vez, estas naves instanciarán cada 1.5s copias de un prefab de “disparo” desde la zona frontal de cada nave. Estos disparos también se trasladarán por sí solos en el eje X negativo global a una velocidad de 10u/s.
2. Se pretende simular un minijuego de recoger huevos que caen desde la zona superior de la pantalla antes de que toquen el suelo. Para ello, se han de instanciar cada 1s y por siempre copias de un sprite de un huevo que caen por su propia gravedad. La coordenada X de instanciación ha de ser random dentro del área de visión de la cámara. En la zona baja del juego tendremos a un personaje controlado por el jugador que tiene como objetivo ir recogiendo los huevos que caen. El jugador no puede salirse de los límites de la pantalla.
 - a. Por cada huevo que recoge, el jugador suma 5 puntos y el huevo en cuestión es destruido. Esta puntuación se va mostrando en el Canvas.
 - b. En cuanto un huevo no sea recogido por el jugador, desactivar el gameObject del jugador y hacer que los huevos no sigan instanciándose.
3. Se pretende desarrollar un pequeño minijuego en el que el jugador va batiendo oleadas de aliens. El player lleva un arma desde la cual ha de poder instanciar ráfagas de copias de un prefab de “bala” y podrá moverse libremente por el escenario (sin salirse de él) con las teclas ‘eje’. Las balas se instanciarán en la zona

frontal del arma (desde el cañon) mientras mantengamos el click izquierdo pulsado. Saldrá una bala cada 0.3s. Las balas se trasladarán por sí solas en la misma dirección en la que apunta el arma.

- a. Consigue que un Spawner instancie copias de un prefab de alien cada 0.4 segundos y por siempre. La zona en la que spawna cada alien será elegida de forma aleatoria entre cuatro puntos posibles localizados en los bordes de la pantalla (izquierda, derecha, arriba y abajo). Estos aliens perseguirán al player por lo que se trasladarán siempre en su dirección.
 - b. Los disparos del personaje matan a los aliens a los 3 toques. Por cada toque, el alien alcanzando por el disparo ha de hacer un parpadeo a rojo. Al último toque, será destruido.
 - c. Si, por otro lado, un alien toca al personaje, este muere inmediatamente (es destruido).
 - d. El jugador va acumulando puntos según el tiempo de juego, siendo visible esta puntuación en un Canvas. Irá sumando 100 puntos por segundo mientras siga vivo.
4. Se pretende desarrollar una pequeña demo de un juego tipo Zelda de GameBoy. El player se encuentra en una sala alargada de una mazmorra. Al comienzo de la sala junto con el player hay una puerta que no se puede abrir. A lo largo de esa sala existen diferentes trampas y al final de esta, un jarrón con una llave. El player tendrá que atravesar dicha sala esquivando las trampas para hacerse con la llave y abrir la puerta inicial.

Hay dos tipos de trampas:

Enemigos vela: Existen 4 enemigos estáticos que escupen bolas de fuego de forma constante y por siempre. Cada **1 segundo** aparecerá una bola de fuego de uno de los 4 elegido al azar. Cada bola saldrá disparada atravesando todo el ancho de la sala.

Discos cortantes: Existen 4 discos cortantes. Cada disco se moverá hacia el lado opuesto de la sala. Una vez alcance dicho destino, se parará de 1 a 2 segundos (aleatorio) para moverse hacia el lado de origen de nuevo, así, sucesivamente y por siempre.

Si el jugador consigue atravesar todas las trampas, llegará al jarrón con la llave. Para destruir este y sacar la llave, hará uso de su **ataque a corta distancia** (con la espada) para romper el jarrón y dejar caer la llave. El ataque puede ser lanzado, por ejemplo, con el click izquierdo del ratón.

Una vez el jarrón se destruya, la llave se **instanciará** en el suelo y podrá ser recogida posicionándose delante de ella y presionando la tecla 'E'.

Una vez el personaje tenga la llave, tendrá que darse la vuelta atravesando las trampas de nuevo y abriendo la puerta que inicialmente estaba cerrada con la tecla 'E'.

El personaje ha de contar con todas las animaciones tipo top-down (moverse, idle y ataque, en las 4 direcciones).